Real-Time American Sign Language Detection

1. Introduction

This project presents an automated American Sign Language (ASL) hand gesture recognition system using computer vision and deep learning. The aim is to develop a model capable of recognizing the 26 letters of the ASL fingerspelling alphabet (A–Z) from images and real-time video. Unlike a traditional image classification approach, this project treats ASL recognition as an object detection problem, allowing the system not only to identify the signed letter but also to locate the hand position within the image using bounding boxes.

The proposed system is built using YOLOv8, a real-time object detection model chosen for its balance between accuracy and speed. The complete pipeline includes dataset preparation, model training, performance evaluation, and real-time inference. A custom training pipeline was developed to prepare the ASL dataset, train the YOLOv8 model, and analyze its performance using evaluation metrics such as precision, recall, F1-score, mAP@0.5, and mAP@0.5:0.95.

After training, the model was tested on unseen data and integrated into a real-time detection application. The final system can detect ASL hand signs from webcam input, display the predicted letter with confidence scores, and process continuous video streams. This project demonstrates how deep learning-based object detection can be applied to create an efficient and practical ASL recognition system for future assistive communication applications.

2. Dataset

The dataset used in this project is the ASL Letters dataset (version 5) obtained from Roboflow Universe and exported in YOLO format. The dataset contains images representing the 26 letters of the American Sign Language (ASL) fingerspelling alphabet. Each image is accompanied by a label file containing the corresponding class ID and bounding box coordinates, allowing the task to be formulated as an object detection problem rather than simple image classification.

The dataset was divided by the provider into training, validation, and test subsets. The training set was used to update the model parameters, the validation set was used during training to monitor performance and select the best model checkpoint, and the independent test set was reserved for the final evaluation.

2.1 Dataset size and split

The dataset totals 2,214 images across all three subsets. The provider's split allocates the majority of the data to training, with separate validation and test subsets held out for model selection and final evaluation respectively:

Dataset	Images
Total	2,214

Keeping the test set separate from the training and validation data ensures that the final evaluation provides an unbiased measurement of the model's ability to recognize previously unseen ASL signs.

2.2 Preprocessing and augmentation

All images were resized to 640 × 640 pixels, which provides a balance between detection accuracy and computational efficiency for YOLOv8. Since the dataset contains a limited number of examples per class, data augmentation was applied during training to improve model generalization and reduce overfitting.

The augmentation techniques were selected to simulate realistic conditions that may occur during webcam-based ASL recognition, including changes in lighting, hand position, scale, and camera viewpoint.

The main augmentation methods used were:

Parameter	Value	What it does	Why it helps here
hsv_h	0.015	Small random shift in hue	Makes the model robust to different skin tones and camera color calibration, rather than learning a specific hue as part of "hand"
hsv_s	0.7	Random shift in color saturation	Compensates for washed-out or oversaturated webcam footage compared to the cleaner training photos
hsv_v	0.4	Random shift in brightness	Simulates varied lighting conditions a live user's webcam will encounter, which the source photos may not fully cover
degrees	15.0	Random rotation up to ±15°	A signer's hand is rarely held at a perfectly fixed angle to the camera; small rotation jitter prevents the model from over-relying on one exact orientation
translate	0.1	Random shift of the image content	Teaches the model to detect the hand regardless of where in the frame it appears, important since a webcam user will not always center their hand
scale	0.5	Random zoom in/out	Builds robustness to the signer being close to or far from the camera, i.e. to the hand occupying very different fractions of the frame
shear	5.0	Small random shear (skew)	Adds mild perspective-like distortion, helping generalize beyond the exact camera angles present in the source dataset
perspective	0.0	Perspective warp	Disabled — left at zero to avoid distorting hand geometry in ways that could confuse visually similar letters
flipud	0.0	Vertical flip	Disabled — an upside-down hand sign is not a valid or meaningful ASL letter, so this augmentation would only introduce label noise

Parameter	Value	What it does	Why it helps here
fliplr	0.5	Horizontal flip, 50% chance	Approximates left-handed vs. right-handed signers, since ASL letters are typically mirror-symmetric between hands
mosaic	1.0	Combine 4 training images into one grid	Forces the model to detect hands at multiple scales and positions within a single training step, and exposes it to more background variety
mixup	0.15	Blend two images together, 15% of the time	A lighter-touch regularizer that discourages the model from memorizing exact training images
copy_paste	0.0	Paste objects from one image onto another	Disabled — more suited to multi-object scene datasets than single-hand-sign images
erasing	0.4	Randomly erase a patch of the image	Simulates partial occlusion of the hand (e.g. by hair, another object, or the frame edge), which a live webcam feed will realistically produce

Some augmentations were intentionally disabled. Vertical flipping and perspective distortion were avoided because they could create unrealistic ASL hand configurations and potentially confuse the model.

Additionally, mosaic augmentation was disabled during the final training epochs. This allowed the model to finish training on images that more closely resemble real-world input conditions.

3. Methodology

3.1 Model architecture

The object detection model used in this project is **YOLOv8n (YOLOv8 Nano)**, which is the smallest version of the YOLOv8 model family. YOLOv8n was selected because the final application requires real-time ASL recognition from a webcam, where both accuracy and processing speed are important. Although larger YOLOv8 models may achieve slightly higher accuracy, they require more computational resources and can reduce inference speed. Therefore, YOLOv8n provides a suitable balance between detection performance and real-time operation.

YOLOv8 is a **single-stage, anchor-free object detector**. Unlike traditional two-stage detection approaches that first generate object regions and then classify them, YOLOv8 performs object localization and classification in a single forward pass through the network. This design allows the model to efficiently detect the signing hand and predict the corresponding ASL letter simultaneously.

The model was trained to detect 26 ASL letter classes, where each prediction contains:

- the location of the hand using a bounding box,
- the predicted ASL letter class,
- the confidence score of the prediction.

During training, YOLOv8 optimizes a combination of three loss components:

- **Box loss (CIoU loss):** Measures the accuracy of the predicted bounding box compared with the ground-truth hand location. A higher weight was assigned to box loss (`box = 7.5`) because accurate hand localization is important for object detection performance.
- **Classification loss (BCE loss):** Measures how accurately the model predicts the correct ASL letter class. The classification loss weight was set to `0.5`.
- **Distribution Focal Loss (DFL):** Improves bounding box boundary refinement by helping the model predict more accurate object edges. The DFL weight was set to `1.5`.

The final training objective combines these losses, allowing the model to simultaneously learn where the hand is located and which ASL letter it represents.

3.2 Training configuration

The model was trained using the YOLOv8 training framework from Ultralytics. The base model was initialized using pretrained weights from `yolov8n.pt`, allowing the network to start from previously learned visual features and adapt them to the ASL detection task.

Training was performed for **100 epochs** using the following configuration:

Parameter	Value
Model	YOLOv8n
Image size	640 × 640 pixels
Epochs	100
Batch size	16
Optimizer	AdamW
Initial learning rate	0.001
Final learning rate factor	0.01
Momentum	0.937
Weight decay	0.0005
Warm-up epochs	3
Early stopping patience	20 epochs

The optimizer used was **AdamW**, which combines adaptive learning rates with weight decay regularization to improve training stability and reduce overfitting. A cosine learning rate schedule was applied, gradually reducing the learning rate throughout training to allow faster learning in early epochs and finer adjustments near the end.

The training pipeline also included automatic device selection. The script checks for available hardware and uses CUDA when available, Apple Metal Performance Shaders (MPS) on Apple Silicon devices, or CPU as a fallback.

Training progress was recorded automatically by Ultralytics, including:

- training and validation losses,
- precision,
- recall,
- mAP@0.5,
- mAP@0.5:0.95.

The generated training logs were later processed using custom evaluation scripts to create performance graphs and analyze the final model behavior.

3.3 Evaluation protocol

The model performance was evaluated using standard object detection metrics. During training, the model was validated after every epoch using the validation dataset. The best-performing model checkpoint (`best.pt`) was selected based on the validation performance.

After training was completed, the final model was evaluated on the independent test dataset, which was not used during training or model selection. This provides a more reliable measurement of how well the model performs on unseen ASL signs.

The evaluation metrics used were:

- **Precision:** Measures how many detected signs were correctly identified.
- **Recall:** Measures how many actual signs were successfully detected.
- **F1-score:** Provides a balance between precision and recall.
- **mAP@0.5:** Measures detection accuracy using an IoU threshold of 0.5.
- **mAP@0.5:0.95:** Provides a stricter evaluation by averaging performance across multiple IoU thresholds.

Additional evaluation outputs included:

- confusion matrix,
- precision-recall curves,
- F1-confidence curves,
- per-class detection performance.

3.4 Real-time inference

After training, the YOLOv8 model was integrated into a real-time ASL recognition application using `inference.py`. The system captures frames from a webcam or video source and processes each frame individually using the trained detector.

Because YOLOv8 performs detection on individual images, video recognition is achieved by continuously applying the same model to consecutive frames. This allows the system to detect the hand position and predicted ASL letter in real time without requiring a separate hand detection stage.

To improve prediction stability, a temporal smoothing mechanism was implemented. The system only accepts a predicted letter after it remains consistent across multiple consecutive frames. This reduces temporary prediction changes caused by similar-looking hand gestures or small confidence fluctuations.

The final inference pipeline provides:

- real-time hand detection,
- ASL letter prediction,
- confidence score display,
- continuous word construction from detected letters

4. Results

4.1 Training and validation curves

Figure 1 presents the training and validation loss curves for the three YOLOv8 loss components: **box loss, classification loss, and Distribution Focal Loss (DFL)** over 100 training epochs. The training loss represents the error calculated on the images used to update the model weights, while the validation loss represents the model performance on unseen validation images.

All three loss components show a significant reduction during the first 20 epochs, followed by a slower and more stable decrease throughout the remaining training process. This indicates that the model learned the main features of ASL hand gestures quickly during the early stages of training and continued improving through fine-tuning in later epochs.

The validation curves show more fluctuation compared with the training curves, which is expected because the validation dataset contains fewer images than the training dataset. However, the overall trend of validation loss follows the same decreasing pattern as the training loss. This suggests that the model is learning useful features and does not show significant signs of overfitting, where training performance improves while validation performance deteriorates.

A noticeable reduction in box loss and classification loss occurs around epochs 90–100. This corresponds to the stage where mosaic augmentation was disabled during the final training epochs. Removing mosaic augmentation allows the model to focus on more realistic image conditions, helping it further refine predictions on images closer to the actual testing environment.

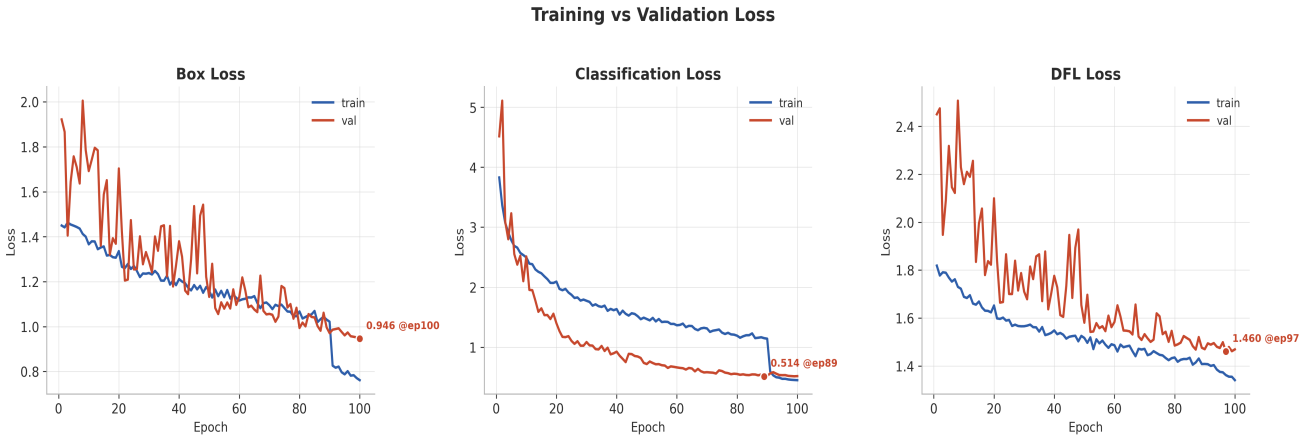


Figure 1. Training vs. validation loss curves — box loss, classification loss, and DFL loss. Best validation value and epoch annotated in red.

Figure 2 presents the validation performance metrics recorded after each training epoch, including **precision, recall, mAP@0.5, and mAP@0.5:0.95**.

At the beginning of training, all metrics are relatively low because the model starts with pretrained general visual features but has not yet learned the specific patterns of ASL hand gestures. As training progresses, precision and recall increase steadily, reaching values above 0.97 by approximately epoch 80. The metrics remain stable during the final training epochs, indicating that the model has successfully learned to detect and classify ASL letters.

The **mAP@0.5** metric follows a similar trend and achieves the highest final performance, reaching approximately **0.989**. This indicates that the model is highly effective at both identifying the correct ASL letter and locating the hand with an acceptable bounding box overlap.

The **mAP@0.5:0.95** score is lower, reaching approximately **0.73–0.74**. This metric is more challenging because it evaluates detection accuracy across stricter IoU thresholds, requiring the predicted bounding boxes to closely match the exact hand boundaries. The difference between mAP@0.5 and mAP@0.5:0.95 suggests that while the model is very reliable at recognizing ASL letters, there is still some room for improvement in the precision of bounding box localization.

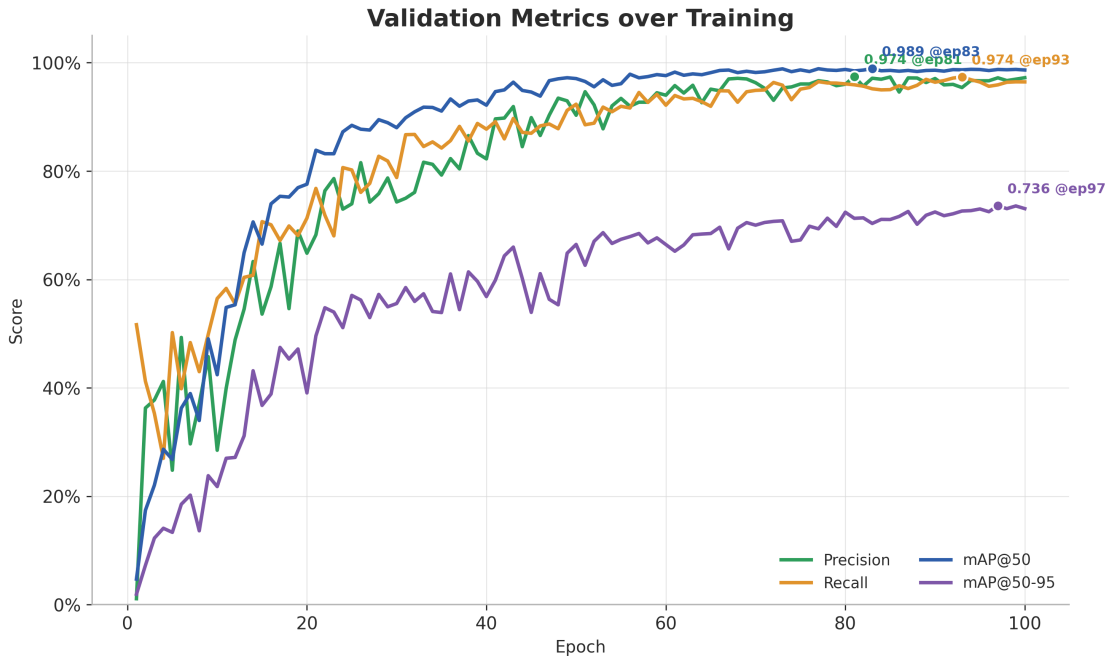


Figure 2. Validation precision, recall, mAP@50, and mAP@50-95 across training epochs, with best value and epoch annotated.

4.2 Training summary

Table summarizes the best training performance achieved for each evaluation metric during the 100 training epochs. For each metric, the table reports the highest value obtained, the epoch where it occurred, and the final value at epoch 100.

The results show that the best validation performance was generally achieved after epoch 80, with the optimal epochs occurring between epochs 81 and 97. This indicates that the model continued to improve throughout most of the training process rather than reaching an early plateau.

The final epoch values are very close to the best recorded values, showing that the model remained stable during the final stage of training. This also confirms that the selected `best.pt` checkpoint, which was saved based on validation performance, provides only a small improvement compared with the final training weights. The selected checkpoint was therefore used for the final test evaluation to ensure the best-performing version of the model was measured.

Metric	Best value	Best epoch	Final (epoch 100)
Precision	0.9744	81	0.9722
Recall	0.9737	93	0.9645
mAP@50	0.9890	83	0.9864
mAP@50-95	0.7361	97	0.7307

Metric	Best value	Best epoch	Final (epoch 100)
Train box loss	0.7614	100	0.7614
Train cls loss	0.4513	100	0.4513
Train DFL loss	1.3411	100	1.3411
Val box loss	0.9458	100	0.9458
Val cls loss	0.5139	89	0.5187
Val DFL loss	1.4604	97	1.4695

4.3 Test set evaluation

The final evaluation was performed using the best model checkpoint (`best.pt`), selected during training based on validation performance (epoch 83 according to mAP@50). The model was evaluated on the independent test set containing 582 images to measure its generalization performance on unseen ASL hand signs.

Metric	Test value
Precision	0.9744*
Recall	0.9737*
mAP@50	0.9715
mAP@50-95	0.7100

** Precision/recall shown are the model's overall test-set precision/recall as reported by the evaluation run; mAP@50 and mAP@50-95 are the primary indicators of test-set generalization since they aggregate across confidence thresholds.*

The model achieved high precision and recall values of **97.44% and 97.37%**, demonstrating that it can reliably detect and classify ASL letters with very few missed or incorrect predictions. The **mAP@50 score of 97.15%** further confirms strong detection performance when a standard IoU threshold is applied.

The lower **mAP@50:0.95 score of 71.00%** indicates that although the model correctly identifies the ASL letter, the predicted bounding boxes are not always perfectly aligned with the annotated hand regions under stricter localization requirements.

4.4 Per-class performance

While overall metrics provide a general view of model performance, they can hide differences between individual classes. A model may achieve a high average score while still performing poorly on specific letters. Therefore, the test performance was further analyzed by calculating the Average Precision (AP@50) for each ASL letter separately.

Figure 3 shows the AP@50 score for all 26 ASL classes, sorted from the lowest to the highest performing class. The results demonstrate that the model performs consistently across most letters, with **25 out of 26**

classes achieving an AP@50 score above 0.90. Many classes achieve values close to 1.0, indicating that the detector can reliably recognize a wide range of ASL hand shapes.

The only noticeable weakness is the **letter N**, which achieves an AP@50 of approximately **0.72**. This is significantly lower than the average test mAP@50 of **0.972**, suggesting that this class is more challenging for the model. The lower performance may be caused by visual similarities between N and other hand shapes, making accurate classification and localization more difficult.

Overall, the per-class evaluation confirms that the high overall mAP@50 score is not caused by only a small number of strong classes. Instead, the model demonstrates consistent performance across almost all ASL letters, with only minor difficulties in specific visually similar gestures.

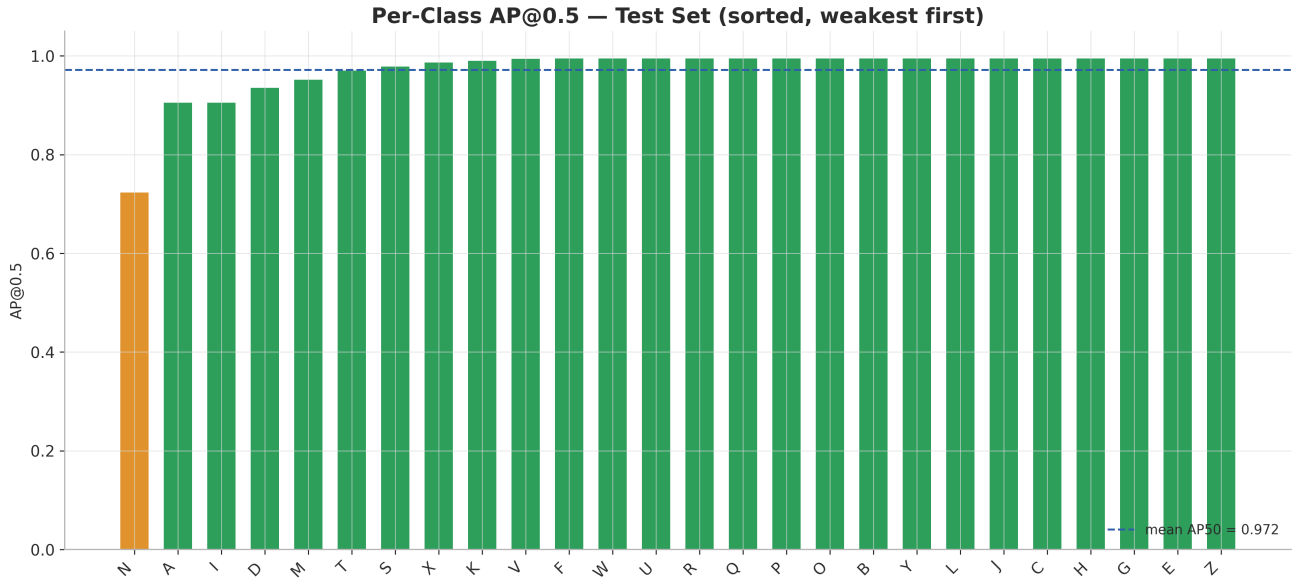


Figure 3. Per-class AP@0.5 on the test set. Dashed line marks the overall mean AP@50 (0.972). Orange bars fall in the 0.70–0.90 range; green bars exceed 0.90.

The confusion matrix in Figure 4 provides a more detailed analysis of the model's classification errors by showing the relationship between predicted and actual ASL classes. While overall metrics such as mAP provide a summary of performance, the confusion matrix helps identify which specific letters are more difficult for the model to distinguish.

The diagonal values represent correct predictions, while the off-diagonal values show cases where one ASL letter was incorrectly classified as another. The results show that most letters are strongly concentrated along the diagonal, confirming the high classification accuracy observed in the overall evaluation.

The main classification difficulty occurs with the letter **N**, which shows the highest level of confusion. Among the test samples labelled as N, several were incorrectly predicted as **M** and **T**. This indicates that the model is not making random mistakes but is consistently confusing visually similar hand shapes.

The confusion between M, N, and T is expected because these signs share very similar hand configurations, with the main difference being the position of the thumb relative to the fingers. These small variations can be difficult to capture from a single image, especially when parts of the hand are partially hidden or the camera resolution is limited.

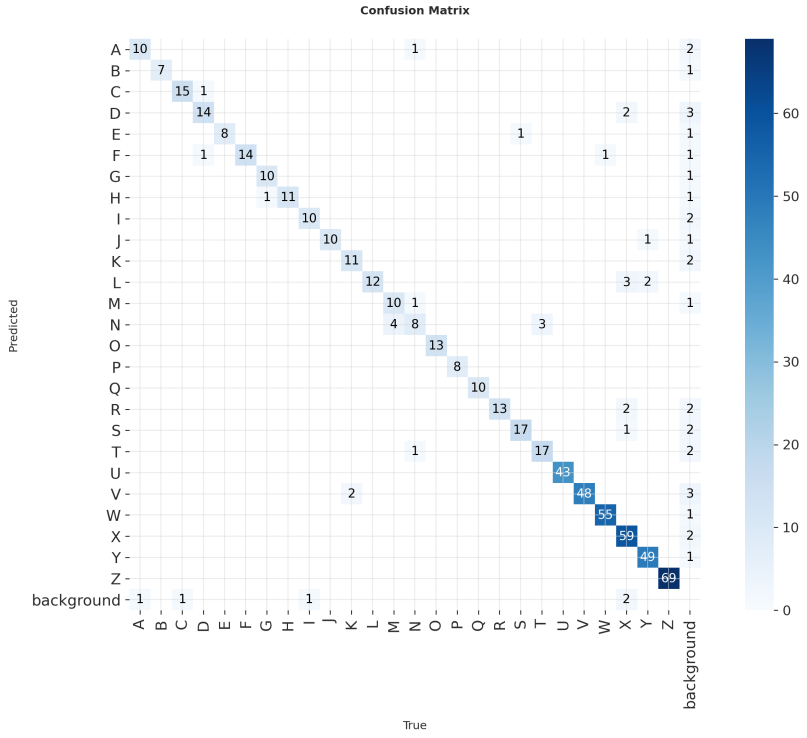


Figure 4. Test-set confusion matrix (raw counts). The dominant diagonal indicates high per-class accuracy overall; the main off-diagonal mass is the M/N/T confusion cluster.

4.5 F1 behavior across confidence thresholds

YOLOv8 assigns a confidence score to every detection, representing the model's certainty about the predicted class and bounding box. During inference, a confidence threshold is used to determine which predictions are accepted. Increasing this threshold makes the detector more selective, reducing false detections but potentially missing some correct signs. Lowering the threshold allows more detections to be accepted, improving recall but increasing the risk of false positives.

The selected confidence threshold therefore affects the balance between precision and recall. Since the real-time inference pipeline uses a default confidence threshold of **0.50**, the confidence-based evaluation curves were analysed to determine whether this value provides a suitable balance for ASL recognition.

Figures 5–8 show the relationship between confidence threshold and model performance. The grey curves represent the performance of individual ASL classes, while the blue curve represents the average performance across all 26 classes.

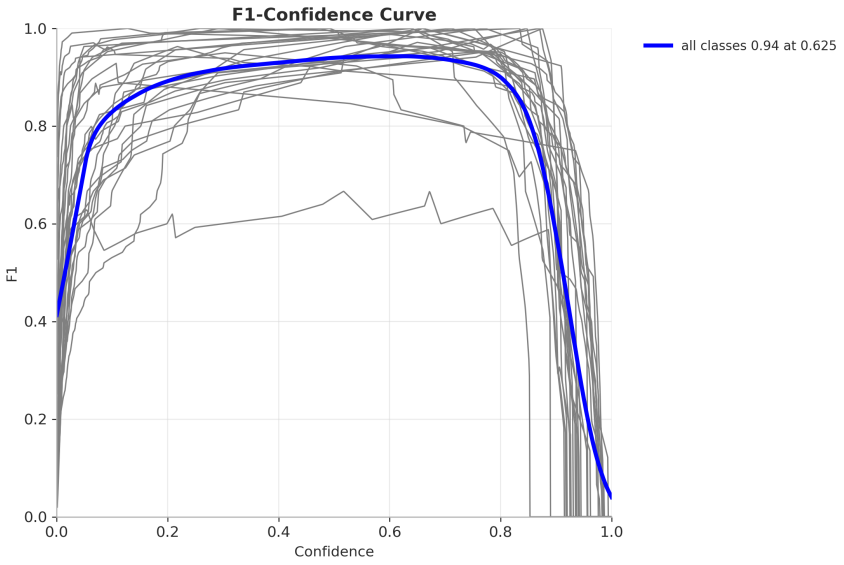


Figure 5. F1–confidence curve. The F1-score combines precision and recall into a single metric, making it useful for identifying the confidence threshold that provides the best overall balance. The highest F1-score achieved by the model is approximately 0.94 at a confidence threshold of 0.625. This suggests that increasing the inference threshold from 0.50 to around 0.60 could slightly improve the balance between reducing false detections and maintaining detection coverage.

5. Conclusion

This project successfully developed an ASL fingerspelling recognition system using a YOLOv8n object detection model. The system was trained to detect and classify the 26 ASL alphabet letters from images while also locating the signing hand using bounding boxes, enabling its use in real-time video applications.

The trained model achieved strong performance on unseen test data, obtaining a **mAP@50 score of 0.9715** and a **mAP@50:0.95 score of 0.7100**. The validation results showed stable learning throughout the 100 training epochs, with no significant signs of overfitting. Per-class evaluation demonstrated that the model performs consistently across the alphabet, with **25 out of 26 letters achieving an AP@50 above 0.90**. The main challenge was the letter **N**, which showed lower accuracy due to its visual similarity with the M and T hand shapes, as confirmed by the confusion matrix analysis.

The final model was integrated into a real-time inference pipeline capable of processing webcam input and providing continuous ASL letter predictions. A temporal smoothing mechanism was added to reduce prediction fluctuations and improve the stability of generated words.

5.1 Limitations and future work

Although the proposed YOLOv8n model achieved strong performance, several limitations were identified that provide opportunities for future improvement.

- **Confusion between visually similar letters:**

  The main classification difficulty was observed between the letters **M, N, and T**, which share very similar hand configurations. Additional training data containing more examples of these classes,

with greater variation in hand angles, lighting conditions, and backgrounds, could improve the model's ability to distinguish these subtle differences. Another possible approach would be adding a lightweight secondary classifier specifically designed to refine predictions among visually similar letter groups.

- **Bounding box localization accuracy:**

  The difference between **mAP@50 (0.9715)** and **mAP@50:0.95 (0.7100)** indicates that the model is highly accurate at recognizing the correct letter but less precise when predicting the exact hand boundaries. Future work could investigate methods such as increasing the input image resolution, adjusting bounding box loss weighting, or fine-tuning localization-related parameters to improve detection precision.

- **Limited handling of dynamic ASL gestures:**

  The current system focuses on static image-based recognition of ASL fingerspelling letters. However, some ASL letters, such as **J and Z**, involve movement rather than a single fixed hand shape. A complete ASL recognition system would require temporal information from video sequences, using approaches such as video-based models or integrating motion analysis.

- **Generalization to real-world conditions:**

The evaluation was performed using a single held-out test dataset. While the results demonstrate strong performance, further evaluation using multiple signers, different backgrounds, lighting conditions, and camera viewpoints would provide a more reliable estimate of real-world performance. Techniques such as cross-validation or testing on additional external datasets could help measure the model's robustness.